Vietnamese Traffic Sign Detection and Warning System

Hoang Gia Bao

Abstract

Accurate and real-time traffic sign recognition is a key component of Advanced Driver Assistance Systems (ADAS) and autonomous vehicles, particularly when deployed on resource-constrained edge devices. However, existing solutions often face challenges in Vietnam's traffic environment due to the lack of datasets that realistically reflect local real-world conditions. To address this issue, I employ data augmentation techniques using the Albumentations library and additional augmentation functions on a base dataset compiled from real-world extracted images to simulate adverse environmental conditions commonly encountered in Vietnam, including low-light nighttime conditions, fog, heavy rain, and motion blur. Additionally, an audio-based warning mechanism is incorporated to provide real-time auditory alerts to drivers upon detecting traffic signs, thereby enhancing the system's practicality, usability, and applicability in real-world driving scenarios. Based on this augmented dataset, I propose a lightweight recognition system built on the YOLO11n architecture, optimized to balance detection accuracy and computational efficiency. Experimental results demonstrate strong performance, achieving a mAP@0.5 of 0.9605, processing speeds of 168.86 FPS on an NVIDIA A100 GPU and 77.52 FPS on a Samsung Galaxy S23 smartphone, with a compact model size of only 5.54 MB. These results confirm the feasibility and effectiveness of the proposed system for real-world applications, paving the way for the development of ADAS solutions tailored to Vietnam's traffic conditions.

Introduction

Traffic sign detection and recognition is a crucial component of Advanced Driver Assistance Systems (ADAS), supporting traffic safety and enabling various levels of autonomous driving. These systems require both high accuracy and low latency to operate reliably in diverse real-world environments. While vehicle and pedestrian detection often demand high frame rates ($\geq$30 FPS), traffic sign recognition – being relatively static - typically requires only about 10 FPS ($<$100 ms latency) to ensure timely driver assistance.

Recent research has applied and optimized object detection models such as YOLO, Faster R-CNN, and MobileNet-SSD for traffic sign detection, especially for small and distant signs under challenging conditions. However, most existing studies rely on foreign datasets or limited Vietnamese datasets that lack diversity in sign types, environmental conditions, and traffic scenarios. As a result, current models may not generalize well to the complex characteristics of Vietnamese highways, national roads, and urban traffic.

This study addresses these limitations by proposing a lightweight and efficient traffic sign detection system tailored for Vietnam. The main contributions include:

1. Custom-designed data augmentation functions and parameters, fine-tuned using the Albumentations library, have been employed to generate images with high fidelity to real-world conditions in Vietnam.

2. Edge-optimized Model: Instead of proposing a new architecture, we focus on practical optimization. We develop a complete pipeline for preprocessing, training, and evaluating lightweight YOLO variants. Six YOLO models (YOLOv8n/s, YOLOv10n/s, YOLO11n/s)

are benchmarked in terms of accuracy, speed, and model size. YOLO11n achieves the best balance (mAP@0.5:0.95 = 0.8022, up to 77.5 FPS on mobile devices, 5.54 MB), making it highly suitable for edge deployment.

3. End-to-end System Validation: We implement and test the entire system on consumer smartphones (Google Pixel 5, Samsung Galaxy S23), confirming its real-world feasibility for ADAS applications.

4. The image/video test runner, built on Streamlit, enables users to verify and recognize traffic signs without the need to download or install an application.

5. A concise audio warning module that delivers brief yet informative alerts, effectively conveying the essential meaning of each detected traffic sign to the driver.

Related work

Traffic sign detection is a fundamental problem in computer vision and plays an important role in Advanced Driver Assistance Systems (ADAS) and autonomous vehicles. With the rapid development of deep learning, Convolutional Neural Network (CNN)-based methods have achieved significant improvements in object detection performance. Early two-stage detectors, such as R-CNN and its successors, provided high detection accuracy but suffered from relatively high computational complexity and slow inference speed. To address these limitations, one-stage detectors, particularly the YOLO (You Only Look Once) family, were introduced to achieve a better balance between detection accuracy and inference efficiency.

The YOLO family has undergone continuous architectural improvements over successive generations. Early versions, from YOLOv1 to YOLOv3, established the foundation for real-time object detection and introduced multi-scale feature representations to improve the detection of small objects. Later generations, including YOLOv5, further improved computational efficiency and provided lightweight variants suitable for deployment on resource-constrained devices. YOLOv8 introduced a modern anchor-free detection architecture and further improvements in both accuracy and inference efficiency. More recently, YOLOv10 incorporated architectural optimizations aimed at reducing inference latency and computational overhead, while YOLO11 further improved the trade-off between accuracy, model complexity, and deployment efficiency.

In our study, we evaluated YOLOv8, YOLOv10, and YOLO11 to identify the most suitable architecture for real-time traffic sign detection on edge devices. Based on the experimental results, we selected the lightweight YOLO11n variant as the final detection model, achieving a compact model size of only 5.54 MB while maintaining high detection performance. This makes the model particularly suitable for deployment on resource-constrained edge platforms.

Another important limitation of existing traffic sign datasets is their inability to fully represent the diverse and challenging environmental conditions encountered in real-world Vietnamese traffic. Publicly available datasets, such as GTSRB and TT100K, primarily focus on traffic environments from Germany and China, respectively, and may not adequately capture local characteristics or adverse visual conditions in Vietnam. To improve the robustness and practical applicability of our system, we constructed a real-world traffic sign dataset from extracted images and applied data augmentation using the Albumentations library and additional augmentation techniques. These transformations were designed to simulate challenging conditions commonly encountered in Vietnam, including low-light nighttime scenes, fog, heavy rain, and motion blur. Through this augmentation strategy, we increased the diversity of the training data and improved the model's ability to generalize under varying real-world conditions.

Despite the growing adoption of lightweight YOLO models, their effectiveness for Vietnamese traffic sign detection and real-time deployment on specific edge devices has not been sufficiently investigated. Therefore, my work focuses not only on detection accuracy but also on

practical deployment efficiency, including inference speed, model size, and real-world robustness under challenging environmental conditions.

Dataset and Proposed Model

Base dataset

The original dataset was obtained from the VR-TSD Computer Vision Dataset, which contains 58 traffic sign categories commonly encountered in Vietnam. The dataset consists of 8,078 images with 13,016 manually annotated bounding boxes. It was originally divided into 70% for training (5,668 images), 16% for validation (1,259 images), and 14% for testing (1,151 images). On average, each image contains one to three traffic signs, while some images contain up to five signs.

During the initial analysis of the dataset, I observed a considerable imbalance in the distribution of samples across traffic sign categories. Several classes contained substantially fewer samples than others, which could lead to biased learning and reduce the model's ability to generalize to underrepresented classes. To mitigate this issue, we introduced class-specific thresholds to identify categories with insufficient representation and selectively increase their contribution during the data augmentation process. The primary purpose of these thresholds was to improve class balance while preventing excessive augmentation of already well-represented categories.

I subsequently applied a diverse set of image augmentation techniques using the Albumentations library and additional custom augmentation functions. These transformations were designed to simulate challenging environmental conditions commonly encountered in real-world Vietnamese traffic, including low-light nighttime conditions, fog, heavy rain, and motion blur. To integrate the augmentation strategy into the existing training pipeline without unnecessarily modifying the underlying implementation, we also employed a monkey-patching approach to extend and control the augmentation behavior. This allowed us to dynamically apply augmentation according to the predefined thresholds and reduce the risk of overfitting to particular augmentation patterns. In particular, the augmentation strategy was designed to prevent the model from memorizing synthetic visual artifacts rather than learning the intrinsic characteristics of traffic signs.

During the experimental process, I observed an unexpected discrepancy between the validation and test results. In particular, the mAP@0.5 obtained on the validation set was 9.38 percentage points lower than that obtained on the test set. Since the test set should normally provide an independent estimate of the model's generalization performance, such a substantial discrepancy raised concerns about a possible data leakage or distribution inconsistency between the validation and test subsets. Although the observed difference alone was insufficient to confirm data leakage, we considered it necessary to re-evaluate the original data split.

Therefore, I merged the validation and test subsets and performed a new random split to obtain a more consistent evaluation protocol. The resulting dataset was divided into 1,410 images for validation and 1,000 images for testing. This reorganization was intended to reduce potential distribution bias between the two evaluation subsets and provide a more reliable assessment of the model's generalization performance.

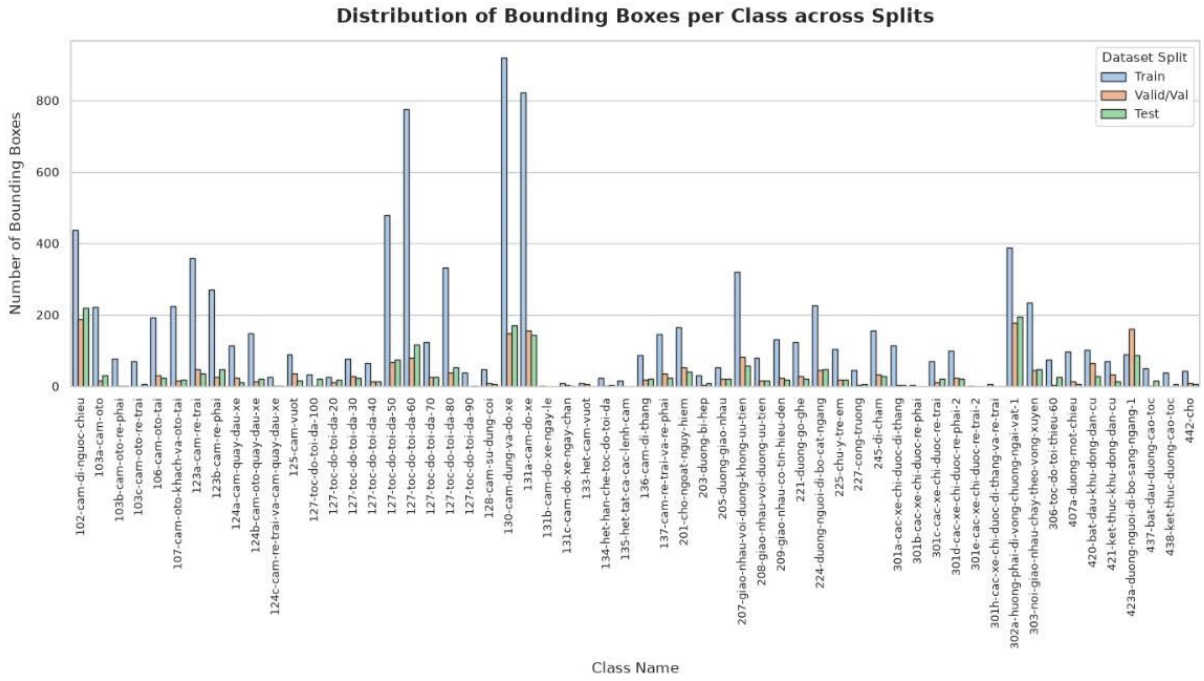

Figure 1: Number of Bounding Boxes of each label in VR-TSD

Methodology

The experimental pipeline was organized into eight stages, covering dataset preparation, class-aware augmentation, model training, post-training threshold optimization, robustness evaluation, edge-device benchmarking, and web deployment. The experiments were conducted in a Kaggle environment with NVIDIA T4 GPU acceleration.

Dataset Preparation: I initialized the experimental environment, configured deterministic seeds, and prepared independent training, validation, and test sets. The dataset was organized into separate image and label directories to maintain a consistent evaluation protocol.

Rare-Class Analysis and Data Augmentation: I analyzed the class distribution in the training set and introduced a rare-class threshold of fewer than 50 instances to identify underrepresented categories. I also included several manually selected difficult classes. Images containing these classes were augmented using bounding-box-aware Albumentations transformations, generating additional training samples while preserving annotation consistency. This process produced 2,289 augmented samples and was designed to improve class balance and reduce overfitting to underrepresented categories.

Evaluation Data Generation: I generated dedicated stress-test datasets from the test set using four challenging conditions: Night, Fog, Rain, and Blur. The transformations were configured at a harder level (+15%) to evaluate the robustness of the trained model under increasingly adverse visual conditions.

Monkey-Patching and Model Training: I used monkey-patching to integrate a custom Albumentations pipeline into the Ultralytics augmentation framework. An A.OneOf selector was employed so that an image received at most one weather effect, preventing unrealistic stacking of multiple environmental transformations. I then trained YOLO11n at an image size of 768 pixels with the configured training parameters.

Per-Class Confidence Threshold Optimization: After training, I optimized the confidence threshold separately for each class using the validation set. The threshold was selected to maximize the F1 score, while classes with fewer than 15 validation instances retained the default confidence threshold of 0.25. This post-training optimization was used to improve

class-specific prediction filtering.

Model Evaluation and Robustness Benchmarking: I evaluated the trained model across three domains: the clean test set, a combined 70/30 dataset containing clean and augmented samples, and individual stress-test datasets for Night, Fog, Rain, and Blur at +15% difficulty. Performance was measured using mAP@0.5, mAP@0.5:0.95, Precision, Recall, and Macro F1 at IoU = 0.5.

Real-Device FPS Validation: I deployed the selected YOLO11n model on a Samsung Galaxy S23 smartphone to measure real-time inference performance and verify its suitability for edge-device deployment.

Web Deployment: Finally, I integrated the trained model into a Streamlit-based web application for practical traffic sign detection. The application supports image and video, together with visual detection and concise audio warnings to improve real-world usability.

3.3 Training and Deployment Environment

All training experiments were conducted on Kaggle using two NVIDIA T4 GPUs. I evaluated YOLOv8, YOLOv10, and YOLO11 to identify a suitable architecture for lightweight traffic sign detection. The final model was based on YOLO11n.

For the YOLO11n training configuration, I used the AdamW optimizer with a batch size of 64, an initial learning rate of 0.0005, and a final learning-rate factor of 0.01. The model was trained for a maximum of 250 epochs with early stopping using a patience of 40 epochs. The input image size was set to 768 × 768 pixels to better preserve features of small traffic signs. A deterministic random seed was also used to ensure reproducibility across experiments.

Table 1: Hyperparameters for model training

Category	**Specifications**	**Details**
Training environment	GPU	2 × NVIDIA T4
	CUDA Version	12.8
Training parameters	Maximum epochs	250
	imgsz	768
	Batch size	64
	Optimizer	AdamW
	Patience (Early stopping)	40
	Initial learning rate (lr0)	0.0005
	Final learning rate fraction (lrf)	0.01
	Horizontal Flip(FlipLR)	0.0

For real-world evaluation, the models were deployed on Ultralytics app with two representative smartphones: the Samsung Galaxy S23 (upper high-end) and the Google Pixel 5 (upper mid-range). Models were exported to TorchScript and subsequently converted into the NCNN framework for mobile deployment. Input videos were processed at 1080p resolution.

4 Experimental Results

4.1 Overall Model Performance

The average evaluation results of the six models on the test set are summarized in Table 2-3. Among the evaluated models, the larger variants, particularly YOLO26s and YOLO11s, achieved relatively high detection performance, with mAP@0.5:0.95 values of 0.8107 and 0.8114, respectively. However, these models have substantially larger model sizes and consequently require more computational resources, resulting in lower inference speeds on both mobile and edge devices. In contrast, YOLO11n achieved a competitive mAP@0.5:0.95 of 0.8022 while maintaining a much smaller model size of 5.54 MB. This favorable balance between detection accuracy, model size, and inference speed makes YOLO11n more suitable for deployment on memory- and resource-constrained edge devices.

Table 2: Results across different versions of the YOLO model

Model	Size (MB)	Macro F1 @ IoU=0.5	Precision	Recall	mAP@50	mAP@50-95
YOLO11n	**5.54**	**0.9295**	**0.9397**	**0.9262**	**0.9605**	**0.8022**
YOLO11s	22	0.9368	0.9327		0.9545	0.8114
YOLO26n	5.3	0.9278	0.9378	0.9241	0.9587	0.7989
YOLO26s	19.84	0.9341	0.9442	0.9297	0.9658	0.8107
YOLOv8s	20.2	0.9031	0.9211	0.8859	0.9434	0.7567
YOLOv8n	5.1	0.8824	0.9076	0.8619	0.9328	0.7214

A comparative analysis further reveals that YOLO26s achieved slightly higher detection accuracy than YOLO11n, with a Macro F1-score of 0.9341 and a mAP@0.5:0.95 of 0.8107, compared with 0.9295 and 0.8022 for YOLO11n, respectively. However, this improvement was relatively marginal, with a difference of only 0.85 percentage points in mAP@0.5:0.95, while YOLO26s has a model size of approximately 19.84 MB, more than 3.5 times larger than YOLO11n. Therefore, the modest gain in detection accuracy does not sufficiently compensate for the increased memory and computational requirements. Consequently, YOLO11n was selected as the final model, as it provides a more favorable trade-off between detection performance, model compactness, and practical edge deployment efficiency.

Table 3: FPS readings run on devices across different versions of YOLO.

Model	GPU A100 ms/img	GPU A100 FPS	Google Pixel 5 ms/img	Pixel 5 FPS	iPhone 11 Pro Max ms/img	iPhone 11 Pro Max FPS
YOLO26n	6.12	163.40	61	16.39	13.8	72.46
YOLO26s	7.04	142.05	138	7.25	17.6	56.82
YOLOv8n	5.92	168.92	59	16.95	12.9	77.52
YOLOv8s	6.01	166.39	141	7.09	16.8	59.52
YOLO11n	**5.54**	**180.51**	**58**	**17.24**	**12.7**	**78.74**
YOLO11s	6.62	151.06	154	6.49	19.4	51.55

In terms of inference efficiency, YOLO11n also maintained a competitive processing speed across the evaluated platforms. Its lightweight architecture reduces computational overhead and makes it more suitable for real-time applications on mobile and edge devices. Although larger models such as YOLO26s may provide slightly higher detection accuracy, their increased computational and memory requirements can limit real-time performance on resource-constrained hardware. This further supports the selection of YOLO11n for the proposed edge-oriented traffic sign detection system.

4.2 Detailed Performance of YOLO11n

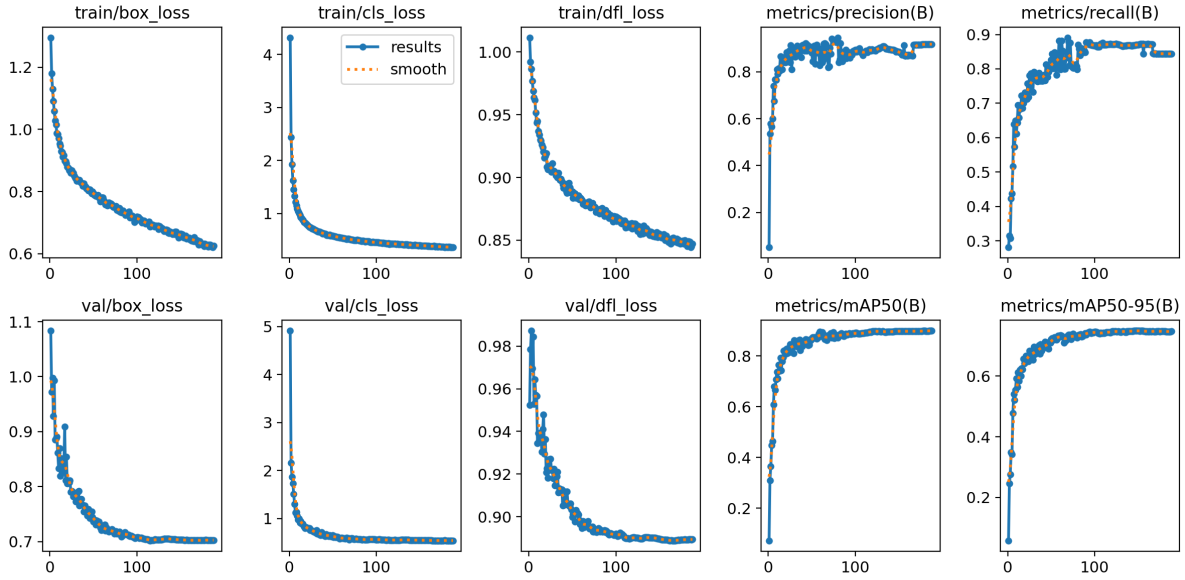

Figure 2: YOLO11n training curve – PR-curve, F1-curve, box/cls/DFL loss per epoch.

The training process of YOLO11n is illustrated in Figure 3. The training losses, including box loss, classification loss, and distribution focal loss, showed a clear decreasing trend throughout the training process. The most substantial reduction occurred during the early epochs, after which the losses gradually decreased and approached stable values toward the end of training. A similar pattern was observed in the validation losses, although several fluctuations were present during the early training stage. These fluctuations became less pronounced as training progressed, and the validation losses eventually stabilized at approximately 0.70 for box loss, 0.55 for classification loss, and 0.89 for DFL loss.

The evaluation metrics exhibited an opposite trend. Precision increased rapidly during the initial training stage and gradually stabilized at approximately 0.92–0.93. Recall also improved substantially from the beginning of training, reaching approximately 0.85 toward the final epochs, although a slight decline can be observed near the end of the training process. Similarly, both mAP@0.5 and mAP@0.5:0.95 increased rapidly in the early epochs and then converged progressively, reaching approximately 0.90 and 0.74, respectively. The convergence of both the loss functions and evaluation metrics indicates that the model progressively learned the target features and reached a relatively stable optimization state. The close correspondence between the training and validation loss trends, together with the stabilization of the evaluation metrics, suggests that the training process was generally stable, without clear evidence of severe overfitting in the later epochs.

Table 4: Robustness Performance of YOLO11n on Combined and Augmented Test Sets

Test Set	mAP@50	mAP@50-95	Precision	Recall	Macro F1
Combined 70/30	0.9563	0.7867	0.9385	0.9152	0.9251
Night (+15%)	0.9402	0.7653	0.9338	0.8973	0.9049
Fog (+15%)	0.9636	0.8003	0.9418	0.9209	0.9269
Rain (+15%)	0.9521	0.7799	0.9111	0.9110	0.9141
Blur (+15%)	0.9314	0.6661	0.9228	0.8752	0.8867

5 Conclusion

This study successfully develops and validates a lightweight, accurate, and real-time traffic sign recognition system specifically designed for Vietnam's road traffic conditions. The proposed VR-TSD dataset contains 58 traffic sign categories covering diverse real-world traffic scenarios.

Through comprehensive experimentation and benchmarking, YOLO11n is identified as the most suitable architecture for edge deployment. Although YOLO26s achieves slightly higher detection performance, YOLO11n provides a better balance between accuracy (mAP@0.5:0.95 = 0.8022) and model compactness (5.54 MB). The application of an optimized confidence threshold further improves the reliability of detection by reducing low-confidence false predictions. In addition, the use of augmented test sets, including Night, Fog, Rain, and Blur conditions, demonstrates that YOLO11n maintains good robustness under environmental variations, with a Macro F1 of 0.9251 on the Combined 70/30 set and 0.9269 under Fog conditions.

Despite these promised results, several limitations remain. The system encounters difficulties in relieved signs that are very distant, almost completely occluded, or auxiliary signs containing textual information. Future work will focus on enriching the dataset with rarer traffic sign types and more adverse environmental conditions, as well as extending system testing to other hardware platforms such as Raspberry Pi and Jetson Nano to ensure broader applicability.

References

[1] Yang, Y., Luo, H., Xu, H., & Wu, F. (2015). Towards real-time traffic sign detection and classification. *IEEE Transactions on Intelligent transportation systems*, 17(7), 2022-2031.

[2] Sarvajcz, K., Ari, L., & Menyhart, J. (2024). Ai on the road: nvidia jetson nano-powered computer vision-based system for real-time pedestrian and priority sign detection. *Applied Sciences*, 14(4), 1440.

[3] Yang, M., & Han, S. (2025). ETS-YOLO: An Efficient YOLO-based Model for Real-Time Traffic Sign Recognition. *Signal, Image and Video Processing*, 19(8), 650.

[4] Chaudhuri, A. (2024). Smart traffic management of vehicles using faster R-CNN based deep learning method. *Scientific Reports*, 14(1), 10357.

[5] Asif, S. M., Anil, T., Tangudu, S., & Keertan, G. S. (2023, May). Traffic Sign Detection Using SSD Mobilenet & Faster RCNN. In *2023 2nd International Conference on Vision Towards Emerging Trends in Communication and Networking Technologies (ViTECoN)* (pp. 1-6).

[6] Huan, T. P., & Lenh, P. P. C. (2024, November). Vietnamese Traffic Sign Detection for Advanced Driving Assistant: Comparison Between YOLOv8 and Faster RCNN. In *International Conference on Advances in Information and Communication Technology* (pp. 617-628). Cham: Springer Nature Switzerland.

[7] Vennelakanti, A., Shreya, S., Rajendran, R., Sarkar, D., Muddegowda, D., & Hanagal, P. (2019, January). Traffic sign detection and recognition using a CNN ensemble. In *2019 IEEE international conference on consumer electronics (ICCE)* (pp. 1-4). IEEE.

[8] Wang, Z., Qu, M., Wang, N., Liu, L., & Su, H. (2025). Lightweight traffic sign detection algorithm based on improved YOLOv5 in snowy environments. *Signal, Image and Video Processing*, 19(5), 1-11.

[9] Jiang, P., Ergu, D., Liu, F., Cai, Y., & Ma, B. (2022). A Review of Yolo algorithm developments. *Procedia computer science*, 199, 1066-1073.

[10] Xuan, Y., Zhang, X. Y., Li, C., Wang, H., & Mu, C. (2025). LAM-YOLOv11 for UAV transmission line inspection: Overcoming environmental challenges with enhanced detection efficiency. *Multimedia Systems*, 31(2), 1-13.

[11] Abdi, N., Parvaresh, F., & Sabahi, M. F. (2024). SeqNet: Sequential networks for one-shot traffic sign recognition with transfer learning. *IEEE Transactions on Intelligent Transportation Systems*.

[12] VR-TSD dataset: https://universe.roboflow.com/vietnam-traffic-sign-recognition-benchmark/vr-tsd

[13] "Source code. https://github.com/HoangGiaBao107/Vietnamese-Traffic-Sign-Detection-and-Warning-System".

[14] "Link website. https://vietnamese-traffic-sign-detection-and-warning-system-hgb.streamlit.app/".